INSTITUTO POLITÉCNICO NACIONAL · ESCUELA SUPERIOR DE CÓMPUTO

Transformación del modelo EER al modelo relacional

Unidad de aprendizaje	Bases de Datos
Práctica	Práctica 3 — Ejercicios 3, 4, 5 y 6
Proyecto	Gestor académico ESCOM
Equipo	Catscoms
Integrantes	Estrada Sánchez Emiliano · Mora Acosta Pablo · Espinosa Gómez David Enrique
Gestor	PostgreSQL 16, base `catscoms_p3`

Este documento recorre el modelo entidad-relación extendido del Ejercicio 4 de la Práctica 2 y lo convierte en tablas, aplicando una por una las nueve reglas de transformación. Para cada elemento del modelo se indica la regla que lo origina y la tabla que resulta. El resultado es el archivo `sql/esquema.sql`, ejecutado sobre el gestor.

Ejercicio 3. Reglas de transformación básicas

Regla 1. Entidades fuertes

Cada entidad fuerte del modelo se convirtió en una tabla, con una columna por atributo simple y su identificador declarado como clave primaria.

Tabla 1. Entidades fuertes y su tabla resultante.

Entidad	Tabla	Clave primaria	Restricciones añadidas
Carrera	`carrera`	`id`	`UNIQUE (nombre, plan_version)`; `CHECK` de que la carga media no sea menor que la mínima
Periodo	`periodo`	`id`	`UNIQUE (nombre)`; `CHECK fecha_fin > fecha_inicio`
Profesor	`profesor`	`id`	—
Rama	`rama`	`id`	`UNIQUE (carrera_id, nombre)`
Estudiante	`estudiante`	`boleta`	`UNIQUE (correo)`; `CHECK` de diez dígitos en la boleta
Materia	`materia`	`id`	`CHECK` de créditos positivos y de unidades de aprendizaje entre 1 y 2

EventoCalendario	evento_calendario	id	CHECK fecha_fin >= fecha_inicio
Resena	resena	id	Ver la sección de la agregación

Una decisión que conviene justificar: `estudiante` usa la **boleta como clave primaria natural** y no una clave subrogada, porque es el identificador oficial que ya emite el Instituto y aparece en kardex, credencial y SAES. Inventar un `id` habría obligado a mantener la boleta como `UNIQUE` de todos modos, con dos identificadores para la misma cosa.

Regla 2. Atributos compuestos

El modelo tiene un atributo compuesto: `Estudiante.nombre`, descompuesto en `nombres`, `ap_paterno` y `ap_materno`. El atributo padre desaparece: no existe ninguna columna `nombre` en `estudiante`.

`ap_materno` es la única de las tres que admite nulos, porque no todos los estudiantes tienen apellido materno registrado y forzarlo produciría cadenas vacías, que son peores que un nulo honesto.

Regla 3. Atributos multivaluados

`Estudiante.telefono` es multivaluado: el procedimiento de reinscripción exige registrar teléfono de casa y teléfono celular antes de iniciar el trámite. Se convierte en tabla propia con la clave de la entidad original más una columna para el valor, y la clave primaria es la combinación de ambas.

```
CREATE TABLE telefono_estudiante (
    estudiante_boleta CHAR(10)    NOT NULL,
    telefono          VARCHAR(15) NOT NULL,
    tipo              VARCHAR(10) NOT NULL,
    CONSTRAINT pk_telefono PRIMARY KEY (estudiante_boleta, telefono),
    CONSTRAINT fk_telefono_estudiante FOREIGN KEY (estudiante_boleta)
        REFERENCES estudiante (boleta) ON DELETE CASCADE
);
```

El `ON DELETE CASCADE` es deliberado: un teléfono sin el estudiante al que pertenece no describe nada.

Ejercicio 4. Transformación de relaciones

Regla 4. Relaciones uno a muchos

La clave del lado uno se propaga como clave foránea al lado muchos. La cardinalidad mínima del lado muchos decide si admite nulos.

Tabla 2. Relaciones uno a muchos y la clave foránea que producen.

Relación	Cardinalidad	Clave foránea	¿Nulos?	Por qué
Carrera — Define — Rama	(1,N):(1,1)	`rama.carrera_id`	No	Toda rama pertenece a una carrera
Carrera — Estudia — Estudiante	(0,N):(1,1)	`estudiante.carrera_id`	No	Nadie está inscrito sin carrera
Carrera — Ofrece — Materia	(1,N):(1,1)	`materia.carrera_id`	No	Toda materia pertenece a un plan
Rama — Agrupa — MateriaOptativa	(1,N):(1,1)	`materia_optativa.rama_id`	No	Ninguna optativa queda huérfana
Profesor — Imparte — Grupo	(0,N):(1,1)	`grupo.profesor_id`	No	Ningún grupo se publica sin titular
Materia — DeMateria — Grupo	(0,N):(1,1)	`grupo.materia_id`	No	Todo grupo imparte exactamente una materia
Periodo — Contiene — EventoCalendario	(0,N):(1,1)	`evento_calendario.periodo_id`	No	El evento pertenece a un periodo
EventoPersonal — Sobre — Materia	(0,1):(0,N)	`evento_personal.materia_id`	**Sí**	La cardinalidad mínima es 0: un recordatorio puede no referirse a ninguna materia

La última fila es la que mejor ilustra la regla: es la única clave foránea del esquema que admite nulos, y lo hace precisamente porque su cardinalidad mínima es 0. Las demás son `NOT NULL` porque su mínimo es 1.

Regla 5. Relaciones muchos a muchos

El modelo tiene dos, y ambas producen tabla propia con clave primaria compuesta.

> **InscripcionPeriodo — Cursa — Grupo**
>
> `(0, N) : (0, N) → tabla cursa`
>
> La relación tiene atributos propios —`calificacion`, `veces_cursada` y `es_adeudo`—, que pasan a columnas. La clave primaria es `(estudiante_boleta, periodo_id, grupo_id)`: las tres juntas, porque la clave de `inscripcion_periodo` ya es compuesta.

> **Materia — Requiere — Materia (recursiva)**
>
> `(0, N) : (0, N) → tabla prerrequisito`
>
> La relación recursiva sobre `Materia` se resuelve con dos claves foráneas a la misma tabla, con nombres de rol

distintos: `materia_id` y `requiere_materia_id`. Un `CHECK` impide que una materia sea prerrequisito de sí misma.

Regla 6. Relaciones uno a uno

Hay una en el modelo: la que une la agregación de `Cursa` con `Resena`, con cardinalidad (0,1):(1,1). Se aplicó la **primera estrategia**: una de las dos entidades es opcional —un curso puede no tener reseña— y la otra obligatoria —toda reseña proviene de un curso—, así que la clave de la obligatoria se propaga a la opcional como foránea con restricción `UNIQUE`.

```
CONSTRAINT uq_resena_por_curso UNIQUE (estudiante_boleta, periodo_id, grupo_id),
CONSTRAINT fk_resena_cursa FOREIGN KEY (estudiante_boleta, periodo_id, grupo_id)
    REFERENCES cursa (estudiante_boleta, periodo_id, grupo_id) ON DELETE CASCADE
```

Se descartó la tercera estrategia —fusionar las dos tablas— aunque a primera vista sería tentador meter las columnas de la reseña dentro de `cursa`. No se hizo por dos razones: la mayoría de las inscripciones no tienen reseña, de modo que la tabla fusionada estaría llena de nulos; y `resena` es propietaria de la entidad débil `reporte_resena`, que necesita una clave a la cual apuntar.

Ese `UNIQUE` es lo que convierte en estructura la regla «una reseña por curso, y solo de quien lo cursó». No hay código de aplicación que pueda saltársela.

Regla 7. Relaciones de orden superior

El modelo no tiene ninguna relación ternaria, y conviene decir por qué, porque el sistema de reseñas parece pedirla. La reseña necesita saber quién la escribió, sobre qué profesor y en qué grupo, lo que sugiere una ternaria Estudiante–Profesor–Grupo. Se descartó por dos defectos: no impediría que alguien reseñara a un profesor con el que nunca coincidió, porque nada obliga a que la terna corresponda a una inscripción real; y el profesor sería redundante, porque el grupo ya determina quién lo imparte.

En su lugar el modelo usa agregación, y al transformarla la verificación deja de ser una regla de aplicación y pasa a ser una consecuencia de la clave foránea.

Ejercicio 5. Entidades débiles y jerarquías

Regla 8. Entidades débiles

El modelo tiene cinco, de los dos tipos de dependencia. La regla las trata distinto: las de identificación llevan clave primaria compuesta por la foránea del propietario más el discriminador; las de existencia conservan su propia clave y añaden la foránea como `NOT NULL`.

Tabla 3. Entidades débiles y la tabla que producen.

Entidad	Propietario	Dependencia	Clave primaria resultante	Borrado
InscripcionPeriodo	Estudiante y Periodo	Identificación	`(estudiante_boleta, periodo_id)`, sin discriminador propio	`CASCADE`
Grupo	Periodo	Identificación	`id` subrogada, con `UNIQUE (periodo_id, codigo)`	`CASCADE`
SesionHorario	Grupo	Identificación	`(grupo_id, dia, bloque)`	`CASCADE`
EventoPersonal	Estudiante	Existencia	`id` propia, con `estudiante_boleta NOT NULL`	`CASCADE`
ReporteResena	Resena	Existencia	`id` propia, con `resena_id NOT NULL`	`CASCADE`

> **Por qué `grupo` lleva clave subrogada.** La regla pide clave compuesta por la foránea del propietario más el discriminador, es decir `(periodo_id, codigo)`. Se conservó esa clave natural declarada como `UNIQUE`, pero la clave primaria es un `id`. El motivo es práctico: `grupo` tiene tres tablas hijas —`sesion_horario`, `cursa` y, a través de ella, `resena`—, y arrastrar dos columnas por cada nivel habría dejado a `resena` con cinco columnas solo de clave foránea. La identidad del dominio sigue protegida por el `UNIQUE`: no pueden existir dos `6CV3` en el mismo periodo.

`SesionHorario` sí conserva la clave compuesta completa, porque no tiene hijas y porque su clave parcial —día más bloque— solo distingue dentro de un grupo: «lunes, bloque 1» describe sesiones de muchos grupos a la vez.

Regla 9. Jerarquía de generalización y especialización

El modelo tiene una: **Materia** se especializa en **MateriaObligatoria** y **MateriaOptativa**, con restricción **disjunta y total**.

Tabla 4. Las tres estrategias frente a este caso.

Estrategia	Qué pasaría aquí	Decisión
A. Tabla por supertipo y por subtipo	Tres tablas. Sin nulos, y `rama_id` puede ser `NOT NULL` dentro de la tabla de optativas. Las consultas que mezclan tipos requieren combinación.	**Elegida**
B. Tabla única con discriminador	Una tabla con cuatro columnas que serían nulas la mitad del tiempo. La	Descartada

	regla «toda optativa tiene rama» no podría declararse: haría falta un `CHECK` condicional del tipo `tipo='OPT' implica rama_id IS NOT NULL`.	
C. Tabla por subtipo, sin supertipo	Dos tablas sin `materia`. Rompe el esquema: `grupo.materia_id` y las dos columnas de `prerrequisito` necesitan apuntar a una sola tabla, y tendrían que duplicarse en dos foráneas excluyentes.	Descartada

La razón decisiva es la tercera fila. Aunque la jerarquía es total y disjunta —el escenario donde el cuadro del enunciado recomienda la estrategia C—, en este modelo el supertipo es destino de claves foráneas desde `grupo` y desde `prerrequisito`. Eliminarlo obligaría a partir esas relaciones en dos, y el gestor ya no podría garantizar que un grupo imparte exactamente una materia.

Cómo se impone la disyunción

Con la estrategia A, nada impediría por sí solo que una misma materia apareciera en las dos tablas de subtipo. Se resolvió sin disparadores, con una clave foránea compuesta:

```
-- en materia
CONSTRAINT uq_materia_id_tipo UNIQUE (id, tipo)

-- en cada subtipo
tipo CHAR(3) NOT NULL DEFAULT 'OBL',
CONSTRAINT ck_mo_tipo CHECK (tipo = 'OBL'),
CONSTRAINT fk_mo_materia FOREIGN KEY (materia_id, tipo)
    REFERENCES materia (id, tipo) ON DELETE CASCADE
```

El subtipo referencia la pareja `(id, tipo)` y fija su propio `tipo` con un `CHECK`. Una materia marcada como `OBL` no puede entrar en `materia_optativa`, porque la pareja `(id, 'OPT')` no existe en `materia`. La disyunción queda garantizada por el gestor y no por la aplicación.

La completitud total, en cambio, no es expresable de forma declarativa: exigiría comprobar que toda fila de `materia` tiene correspondencia en alguno de los dos subtipos, lo que en SQL estándar requiere un disparador o una aserción. Se documenta como regla de negocio verificada en la carga.

Ejercicio 6.1. Esquema relacional completo

Dieciocho tablas. Las claves primarias van subrayadas en la notación habitual; aquí se listan con su tipo de dependencia y la regla que las originó.

Tabla 5. Las dieciocho tablas del esquema.

Tabla	Origen	Regla
`carrera`	Entidad fuerte Carrera	1
`periodo`	Entidad fuerte Periodo	1
`profesor`	Entidad fuerte Profesor	1
`rama`	Entidad fuerte Rama	1 y 4
`estudiante`	Entidad fuerte Estudiante, con el nombre descompuesto	1 y 2
`telefono_estudiante`	Atributo multivaluado `telefono`	3
`materia`	Supertipo de la jerarquía	1 y 9
`materia_obligatoria`	Subtipo	9
`materia_optativa`	Subtipo	9 y 4
`prerrequisito`	Relación recursiva Requiere	5
`inscripcion_periodo`	Entidad débil por identificación	8
`grupo`	Entidad débil por identificación	8 y 4
`sesion_horario`	Entidad débil por identificación	8
`cursa`	Relación muchos a muchos con atributos	5
`resena`	Entidad anclada a la agregación de Cursa	6
`reporte_resena`	Entidad débil por existencia	8
`evento_calendario`	Entidad fuerte EventoCalendario	1 y 4
`evento_personal`	Entidad débil por existencia	8 y 4

El detalle columna por columna, con tipos y restricciones, está en `diccionario-de-datos.pdf`, y el DDL completo en `sql/esquema.sql`.

Reglas de negocio que no caben en una restricción declarativa

Dos límites del modelo no son expresables como cardinalidad ni como `CHECK`, y se implementaron con disparadores. Vale la pena señalarlos porque son justamente donde la transformación mecánica se queda corta.

1. **El tope de carga.** Un estudiante no puede inscribir más unidades de aprendizaje que la carga media de su carrera. El tope no es una constante: vale 7 en ISC y 6 en IIA y LCD, vive en `carrera.carga_media` y se cuenta en unidades de aprendizaje, no en grupos, porque Trabajo Terminal vale dos. Un `CHECK` no puede consultar otras tablas, así que se implementó con el disparador `tg_valida_carga`.

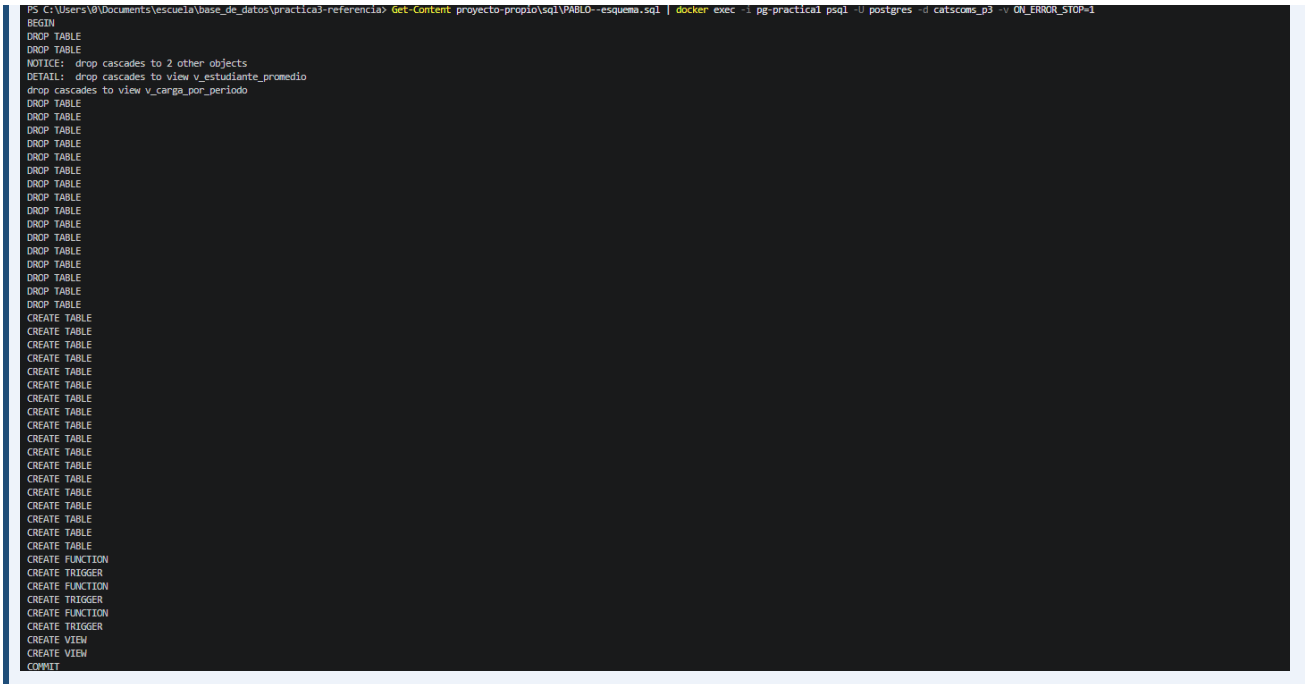
2. **El cupo del grupo.** Mismo caso: el límite depende de `grupo.cupo`, distinto en cada grupo, y hay que contar las filas ya existentes. Disparador `tg_valida_cupo`.

Escribir estos dos topes como cardinalidad —por ejemplo `(0, carga_media)`— habría dado una falsa sensación de rigor: el gestor no habría validado nada.

Ejercicio 6.2. Ejecución del esquema

El archivo `sql/esquema.sql` se ejecutó sobre la base `catscoms_p3`, levantada en el contenedor `pg-practica1` con el `compose.yaml` del repositorio.

```
docker compose up -d
docker exec -it pg-practica1 psql -U postgres -c "CREATE DATABASE catscoms_p3;"
docker exec -i pg-practica1 psql -U postgres -d catscoms_p3 -v ON_ERROR_STOP=1 \
    < proyecto-propio/sql/esquema.sql
```



Al terminar, la base contiene dieciocho tablas y dos vistas.

```
docker exec -it pg-practica1 psql -U postgres -d catscoms_p3 -c "\dt"
```

```
PS C:\Users\0\Documents\escuela\base_de_datos\practica3-referencia> docker exec -it pg-practica1 psql -U postgres -d catscoms_p3 -c "\dt"
              List of relations
 Schema |        Name         | Type  |  Owner
--------+---------------------+-------+----------
 public | carrera             | table | postgres
 public | cursa               | table | postgres
 public | estudiante          | table | postgres
 public | evento_calendario   | table | postgres
 public | evento_personal     | table | postgres
 public | grupo               | table | postgres
 public | inscripcion_periodo | table | postgres
 public | materia             | table | postgres
 public | materia_obligatoria | table | postgres
 public | materia_optativa    | table | postgres
 public | periodo             | table | postgres
 public | prerrequisito       | table | postgres
 public | profesor            | table | postgres
 public | rama                | table | postgres
 public | reporte_resena      | table | postgres
 public | resena              | table | postgres
 public | sesion_horario      | table | postgres
 public | telefono_estudiante | table | postgres
(18 rows)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug pg-practica1
    Learn more at https://docs.docker.com/go/debug-cli/
```

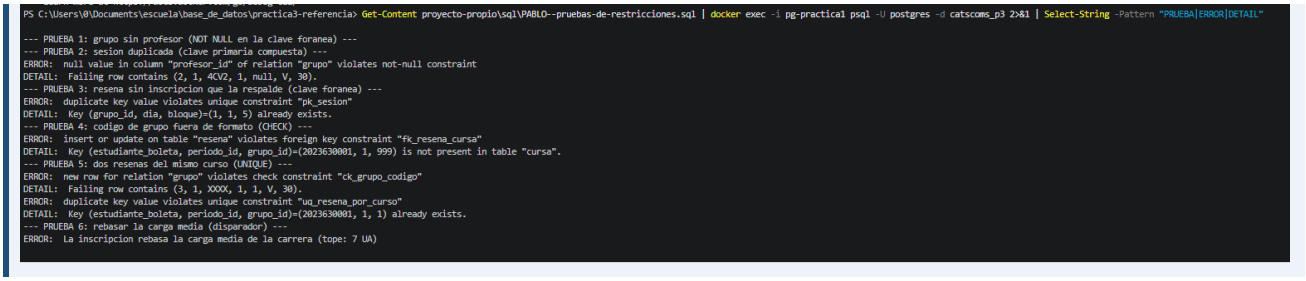
Inserciones que el gestor rechaza

El archivo `sql/pruebas-de-restricciones.sql` carga unos datos mínimos y después intenta seis inserciones inválidas. Cada una viola un tipo distinto de restricción, y el rechazo es la prueba de que el esquema protege los datos.

Tabla 6. Las seis pruebas y el error que devuelve el gestor.

#	Qué se intenta	Qué lo impide	Mensaje de PostgreSQL
1	Publicar un grupo sin profesor titular	`NOT NULL` en la clave foránea	`null value in column "profesor_id" violates not-null constraint`
2	Dos sesiones el mismo día y bloque en un grupo	Clave primaria compuesta	`duplicate key value violates unique constraint "pk_sesion"`
3	Reseñar un grupo que nunca se cursó	Clave foránea a `cursa`	`violates foreign key constraint "fk_resena_cursa"`
4	Código de grupo fuera del formato de la Escuela	`CHECK` con expresión regular	`violates check constraint "ck_grupo_codigo"`
5	Dos reseñas del mismo curso	`UNIQUE` sobre la clave de `cursa`	`duplicate key value violates unique constraint "uq_resena_por_curso"`
6	Inscribir por encima de la carga media	Disparador `tg_valida_carga`	`La inscripcion rebasa la carga media de la carrera (tope: 7 UA)`

```
docker exec -i pg-practica1 psql -U postgres -d catscoms_p3 \
    < proyecto-propio/sql/pruebas-de-restricciones.sql
```



La sexta prueba es la que mejor cierra el argumento del documento: el tope de carga no se puede declarar como cardinalidad ni como `CHECK`, pero sí se puede hacer cumplir, y el modelo lo hace.

Ejercicio 6.5. Preservación del significado

Contra el inventario del Ejercicio 2, ningún elemento del modelo EER se perdió en la transformación.

Tabla 7. Correspondencia entre el inventario y el esquema.

Elemento del modelo EER	Dónde quedó
8 entidades fuertes	8 tablas, cada una con su clave primaria
5 entidades débiles	5 tablas, con el tipo de dependencia reflejado en la forma de la clave
Jerarquía disjunta y total	3 tablas, con la disyunción impuesta por la clave foránea compuesta
Atributo compuesto `nombre`	3 columnas en `estudiante`; el padre desaparece
Atributo multivaluado `telefono`	Tabla `telefono_estudiante` con clave compuesta
Atributos derivados	Vistas `v_estudiante_promedio` y `v_carga_por_periodo`; no se almacenan
Relación recursiva de prerrequisitos	Tabla `prerrequisito` con dos foráneas a `materia`
Agregación sobre `Cursa`	`UNIQUE` y clave foránea compuesta en `resena`
Cardinalidades mínimas	`NOT NULL` cuando el mínimo es 1; nulos permitidos cuando es 0
Topes de carga y cupo	Disparadores, por no ser expresables de forma declarativa

Qué se gana y qué se pierde

Se gana que las reglas dejan de depender del código. «Ningún grupo sin titular», «una reseña por curso», «solo reseña quien cursó» y «dos grupos `6CV3` de periodos distintos son cosas distintas» quedan garantizadas por el gestor, y ninguna aplicación mal escrita puede violarlas.

Se pierde legibilidad directa del modelo. En el diagrama de Chen se ve de un vistazo que `Resena` cuelga de una agregación; en el esquema relacional eso es una clave foránea compuesta con un `UNIQUE`, y hay que saber leerlo. Lo mismo con la jerarquía: el triángulo con los indicadores *d* y *t* se convierte en una pareja `(id, tipo)` que no se interpreta sola. Por eso el diccionario de datos documenta columna por columna qué regla del dominio representa cada restricción.

Y se pierde la completitud total de la jerarquía, que es la única restricción del modelo EER que no quedó implementada de forma declarativa.

Referencias

Elmasri, R., y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Escuela Superior de Cómputo. (2026). *Procedimiento de reinscripción, periodo escolar 2027/1*. Instituto Politécnico Nacional.

Instituto Politécnico Nacional. (2011). *Reglamento General de Estudios*. https://www.ipn.mx/assets/files/normatividad/docs/reglamentos/rge-13-06-2011.pdf

The PostgreSQL Global Development Group. (2025). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/